

SmartLogix

Documentación FullStack.

Integrantes:

Rubén Velásquez.

Maximiliano Millacaris.

Alejandro Francis.

Tabla de Contenidos.

1. Visión General de la Arquitectura
2. Mapa de Microservicios y Puertos.
3. API Gateway.
4. Servicio Usuario (Puerto 8081).
5. Servicio Pedido (Puerto 8082).
6. Servicio Envío (Puerto 8083).
7. Servicio Inventario (Puerto 8084).
8. Servicio Notificaciones (Puerto 8085).
9. Flujo de Comunicación entre Servicios.
10. Pruebas Unitarias.
11. Docker y Contenedores.

1. Visión General de la Arquitectura.

SmartLogix es un sistema de gestión logística construido sobre una arquitectura de microservicios. Cada servicio es completamente independiente, tiene su propia base de datos MySQL y se comunica con los demás exclusivamente a través de HTTP REST. El único punto del sistema es el API Gateway, que enruta las peticiones y valida la seguridad mediante tokens JWT.

La elección de microservicios frente a un monolito permite escalar horizontalmente sólo los servicios que lo requieren, desplegar cambios de forma aislada y contener los fallos. Por ejemplo, si el servicio de notificaciones cae, los pedidos siguen procesándose con normalidad.

Herramientas tecnológicas principales:

- Spring Boot 3 + Java 21 (LTS).
- Spring Security + JWT (HMAC-SHA256) para autenticación y autorización.
- Spring Data JPA + Hibernate para persistencia.
- OpenFeign para comunicación inter-servicios.
- MySQL 8.0 como motor de base de datos.
- Docker + Docker Compose para orquestación de contenedores.
- Swagger para documentación de endpoints.

2. Mapa de Microservicios y Puertos-

El sistema está compuesto por 6 contenedores de aplicación y 5 bases de datos independientes, todos corriendo dentro de la red Docker interna spring-net:

Servicio	Puerto App	Puerto BD	Base de Datos
API Gateway	9090	—	—
serviciousuario	8081	3307	usuarios_db
serviciopedido	8082	3308	pedidos_db
servicioenvio	8083	3309	envios_db
servicioinventario	8084	3310	inventario_db
servicionotificacion	8085	3311	notificaciones_db

Cada base de datos expone un puerto externo diferente (3307–3311) para evitar conflictos con una instalación local de MySQL que ocupa el 3306 por defecto. Internamente, los servicios se comunican usando el nombre del contenedor Docker, no localhost.

3. API Gateway (Puerto 9090).

El API Gateway actúa como la puerta de entrada única al sistema. Ningún microservicio está expuesto directamente al exterior, toda petición HTTP debe pasar primero por aquí. Esto centraliza la seguridad, el enrutamiento y las políticas de acceso.

3.1 Responsabilidades.

- Enrutar las peticiones a cada microservicio según el path de la URL.
- Validar el token JWT en cada petición entrante antes de redirigirla.
- Aplicar control de acceso basado en roles (RBAC).
- Gestionar CORS para permitir peticiones desde el frontend.

3.2 Clases Principales.

JwtUtil.java:

Lee y valida tokens JWT. Extrae el correo y el rol del usuario. Los tokens están firmados con HMAC-SHA256, lo que garantiza que no pueden ser alterados sin conocer la clave secreta.

JwtFilter.java:

Filtro de Spring Security que se ejecuta en cada petición HTTP. Lee el header Authorization: Bearer <token>, invoca JwtUtil para validarlo, y si es válido registra al usuario en el contexto de seguridad con su rol como ROLE_<ROL>.

SecurityConfig.java | Tabla de permisos.

Ruta	Método	Rol Requerido
/usuarios/crear	POST	Público
/usuarios/login	POST	Público
/pedidos/crear	POST	OPERADOR
/pedidos/ver	GET	ADMIN, OPERADOR, LOGISTICA, BODEGA
/pedido/cambiar estado	PUT	LOGÍSTICA
/envios/ver	GET	ADMIN, LOGÍSTICA
/envios/crear	POST	LOGÍSTICA
/productos/crear	POST	BODEGA
/productos/ver	GET	ADMIN, BODEGA, OPERADOR, LOGISTICA
/productos/mod/eliminar	PUT/DELETE	BODEGA

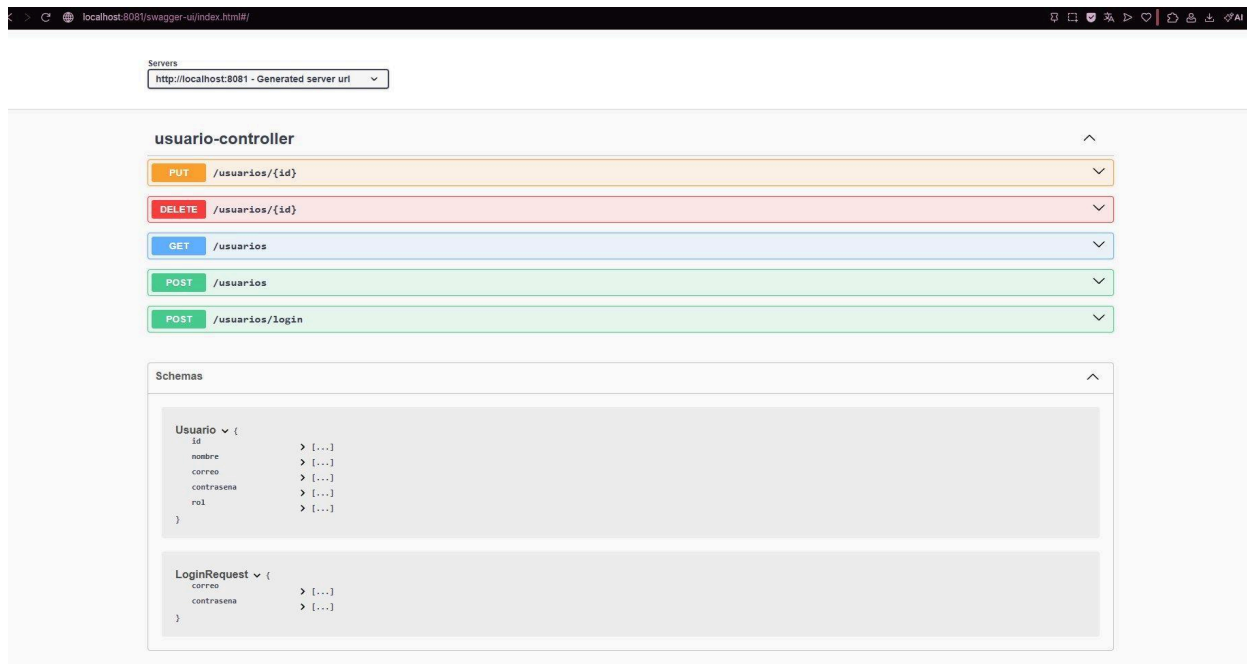
CorsConfig.java:

Configura Cross-Origin Resource Sharing para permitir peticiones desde cualquier origen , soportando GET, POST, PUT y DELETE. Necesario para que el frontend pueda consumir la API desde un dominio diferente.

4. Servicio Usuario (Puerto 8081).

Gestiona el ciclo de vida completo de los usuarios: registro, consulta, actualización, eliminación y autenticación. Es el único servicio que genera tokens JWT, verificados luego por el API Gateway.

4.1 Captura Swagger – Endpoints.



La interfaz Swagger en localhost:8081 muestra los 5 endpoints del usuario-controller: PUT y DELETE para gestión por ID, GET para listar, POST para crear y POST /usuarios/login para autenticar. El esquema inferior muestra los campos Usuario y el objeto LoginRequest.

4.2 Endpoints REST /usuarios.

Método	Ruta	Descripción
POST	/usuarios	Crear un nuevo usuario en el sistema
GET	/usuarios	Listar todos los usuarios registrados
PUT	/usuarios/{id}	Actualizar datos de un usuario existente
DELETE	/usuarios/{id}	Eliminar un usuario por su ID
POST	/usuarios/login	Autenticar usuario y retornar token JWT

4.3 Entidad Usuario.

```
{ id, nombre, correo, contraseña (write-only), rol }
```

Los roles disponibles son: ADMIN, OPERADOR y LOGISTICA. El campo contraseña está marcado como write-only, nunca se devuelve en respuestas (medida de seguridad básica).

4.4 Lógica de Negocio (UsuarioServiceImpl).

- Nombre: mínimo 3 caracteres, solo letras y espacios (validado con regex).
- Correo: solo se aceptan dominios gmail.com, hotmail.com, duoc.cl y duocuc.cl.
- Contraseña: entre 8 y 24 caracteres.
- Al crear: la contraseña se hashea con BCrypt antes de persistir.
- Al actualizar: si no se envía contraseña, se conserva el hash anterior.
- El correo se normaliza siempre a minúsculas antes de guardar.

4.5 Generación del Token JWT.

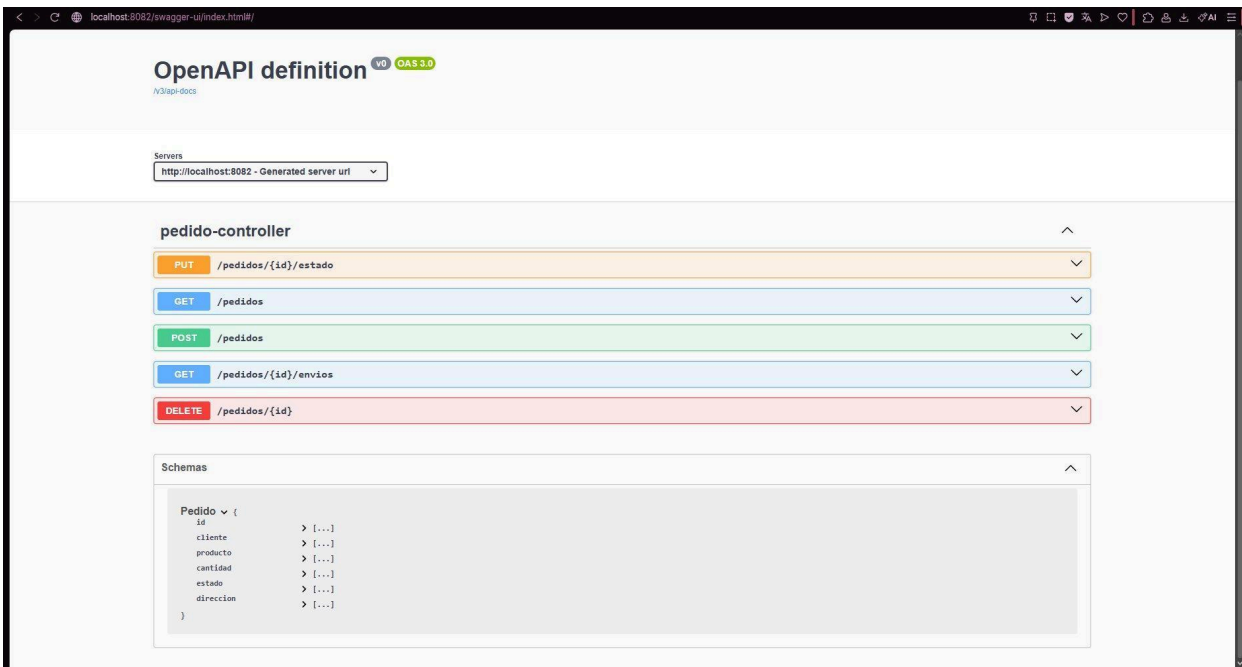
El token incluye el correo como subject y el rol como claim personalizado. Firmado con HMAC-SHA256 y con expiración de 1 hora. Debe enviarse en el header Authorization de todas las peticiones subsiguientes.

Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...

5. Servicio Pedido (Puerto 8082).

El servicio de pedidos es el más complejo del sistema. Al crear un pedido, verifica stock en inventario, lo descuenta, persiste el pedido y crea automáticamente un envío. Esta orquestación se realiza mediante Feign Clients.

5.1 Captura Swagger – Endpoints.



La vista Swagger de localhost:8082 muestra el pedido-controller con 5 endpoints: PUT para cambiar estado, GET para listar y ver envíos asociados, POST para crear y DELETE para eliminar. El esquema Pedido confirma sus campos: cliente, producto, cantidad, estado y dirección.

5.2 Endpoints REST /pedidos.

Método	Ruta	Descripción
POST	/pedidos	Crear pedido (verifica stock + crea envío automático).
GET	/pedidos	Listar todos (filtro opcional ?cliente=).
PUT	/pedidos/{id}/estado	Actualizar estado del pedido.
GET	/pedidos/{id}/envios	Ver pedido con todos sus envíos asociados.
DELETE	/pedidos/{id}	Eliminar pedido y sus envíos relacionados.

5.3 Entidad Pedido.

```
{ id, cliente, producto, cantidad, estado, direccion }
```

5.4 Flujo Completo de Creación de Pedido.

Al recibir un POST /pedidos, el servicio ejecuta los siguientes pasos en orden:

Paso	Acción	Si Falla
1	Valida campos: producto, cantidad > 0, dirección y cliente no vacíos	Excepción 400.
2	Llama a InventarioClient.hayStock()	Excepción: sin stock.
3	Llama a InventarioClient.descontarStock()	Excepción: error al descontar.
4	Persiste el pedido con estado PENDIENTE	Error BD.
5	Llama a EnvioClient.crearEnvio() con la dirección	Log en consola; pedido NO se revierte.

5.5 Feign Clients.

InventarioClient → <http://servicioinventario:8084>

[GET] /productos/stock?nombre=&cantidad – Verifica disponibilidad de stock.

[PUT] /productos/descontar?nombre=&cantidad – Descuenta unidades del inventario.

EnvioClient → <http://servicioenvio:8083>

[GET] /envios/pedido/{pedidoId} – *Obtiene envíos de un pedido.*

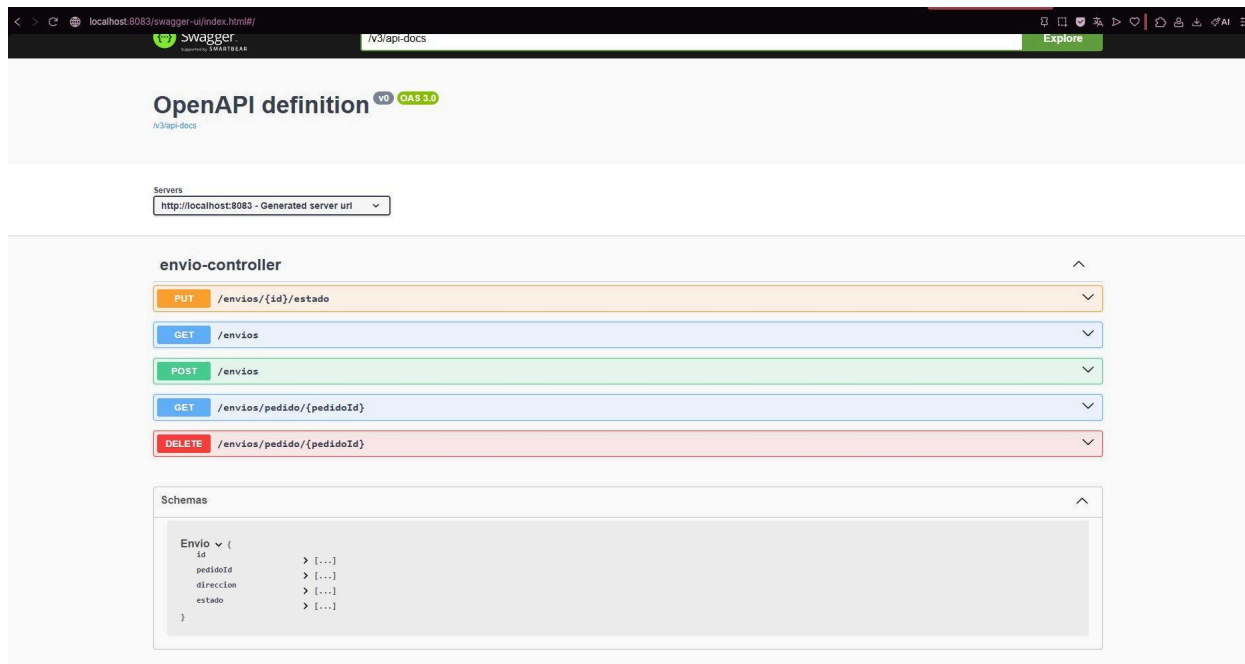
[POST] /envios — *Crea nuevo envío.*

[DELETE] /envios/pedido/{pedidoId} – Elimina envíos al borrar un pedido.

6. Servicio Envío (Puerto 8083).

Gestiona el ciclo de vida de los envíos asociados a los pedidos. Los envíos nacen automáticamente al crear un pedido, pero también pueden gestionarse de forma independiente a través de este servicio.

6.1 Captura Swagger – Endpoints



La pantalla Swagger de localhost:8083 muestra el envio-controller organizado en dos recursos: el envío individual (por id) para actualizar estado, y los envíos agrupados por pedido (pedidoId) para consultar y eliminar. El esquema confirma los campos: id, pedidoId, direccion y estado.

6.2 Endpoints REST /envios

Método	Ruta	Descripción
POST	/envios	Crear envío (estado inicial automático: PENDIENTE)
GET	/envios	Listar todos los envíos
GET	/envios/pedido/{pedidoId}	Obtener envíos de un pedido específico
PUT	/envios/{id}/estado	Actualizar el estado de un envío
DELETE	/envios/pedido/{pedidoId}	Eliminar todos los envíos de un pedido

6.3 Entidad Envío.

```
{ id, pedidoId, direccion, estado }
```

6.4 Máquina de Estados del Envío.

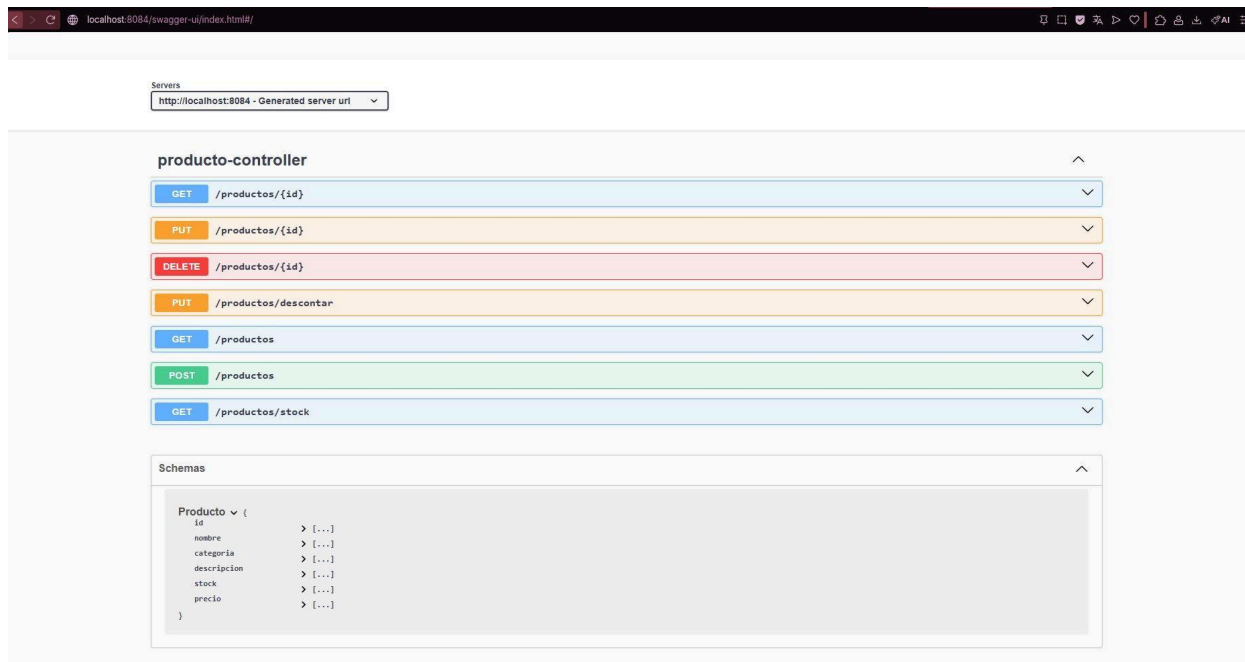
Los únicos valores válidos para el campo estado son los siguientes. Cualquier otro valor provoca una excepción con mensaje descriptivo:

Estado	Descripción
PENDIENTE	Estado inicial al crear el envío. Aún no ha salido del almacén.
EN_CAMINO	El paquete está en tránsito hacia el destinatario.
ENTREGADO	El paquete llegó exitosamente al cliente final.

7. Servicio Inventario (Puerto 8084).

Gestiona el catálogo de productos y el control de stock. Actúa como fuente de verdad sobre la disponibilidad de mercancías. El servicio de pedidos lo consulta antes de aceptar cualquier orden.

7.1 Captura Swagger – Endpoints.



La pantalla Swagger de localhost:8084 muestra el producto-controller con 7 endpoints. Destacan GET /productos/stock para verificar disponibilidad y PUT /productos/descontar para rebajar unidades, que son los consumidos por el Feign Client del servicio de pedidos. El esquema muestra los campos completos del modelo Producto.

7.2 Endpoints REST /productos.

Método	Ruta	Descripción
POST	/productos	Crear nuevo producto en el catálogo.
GET	/productos	Listar todos los productos.
GET	/productos/{id}	Buscar producto por ID.
PUT	/productos/{id}	Actualizar datos de un producto.
DELETE	/productos/{id}	Eliminar producto del catálogo.
GET	/productos/stock?nombre=&cantidad=	Verificar si hay stock suficiente.
PUT	/productos/descontar?nombre=&cantidad=	Descontar unidades del stock.

7.3 Entidad Producto.

```
{ id, nombre, categoria, descripcion, stock, precio }
```

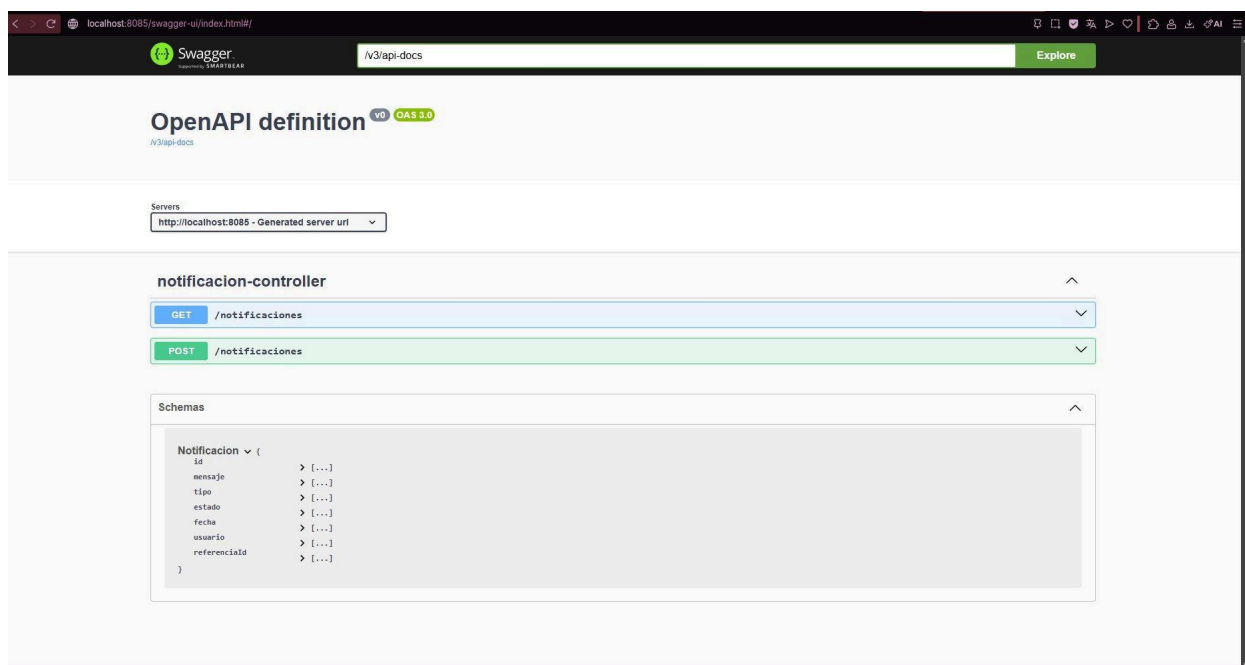
7.4 Validaciones de Negocio.

- El nombre y categoria son campos obligatorios (no pueden estar vacíos).
- El stock no puede ser un valor negativo.
- El precio debe ser mayor a 0.
- La búsqueda para descontar stock se realiza por nombre del producto, no por ID.

8. Servicio Notificaciones (Puerto 8085).

Servicio de registro de eventos del sistema. Actúa como log de auditoría: almacena mensajes sobre lo que ocurre con metadatos como tipo, usuario que generó el evento y referencia al recurso involucrado.

8.1 Captura Swagger – Endpoints.



La vista Swagger de localhost:8085 muestra el notificacion-controller: solo dos endpoints. No hay operaciones de actualización ni eliminación, convirtiéndolo en un registro append-only. El esquema muestra los 7 campos de la entidad, donde estado y fecha se asignan automáticamente sin que el cliente los envíe.

8.2 Endpoints REST /notificaciones.

Método	Ruta	Descripción
POST	/notificaciones	Crear nueva notificación/evento en el sistema
GET	/notificaciones	Listar todas las notificaciones registradas

8.3 Entidad Notificacion.

```
{ id, mensaje, tipo, estado, fecha, usuario, referenciaId }
```

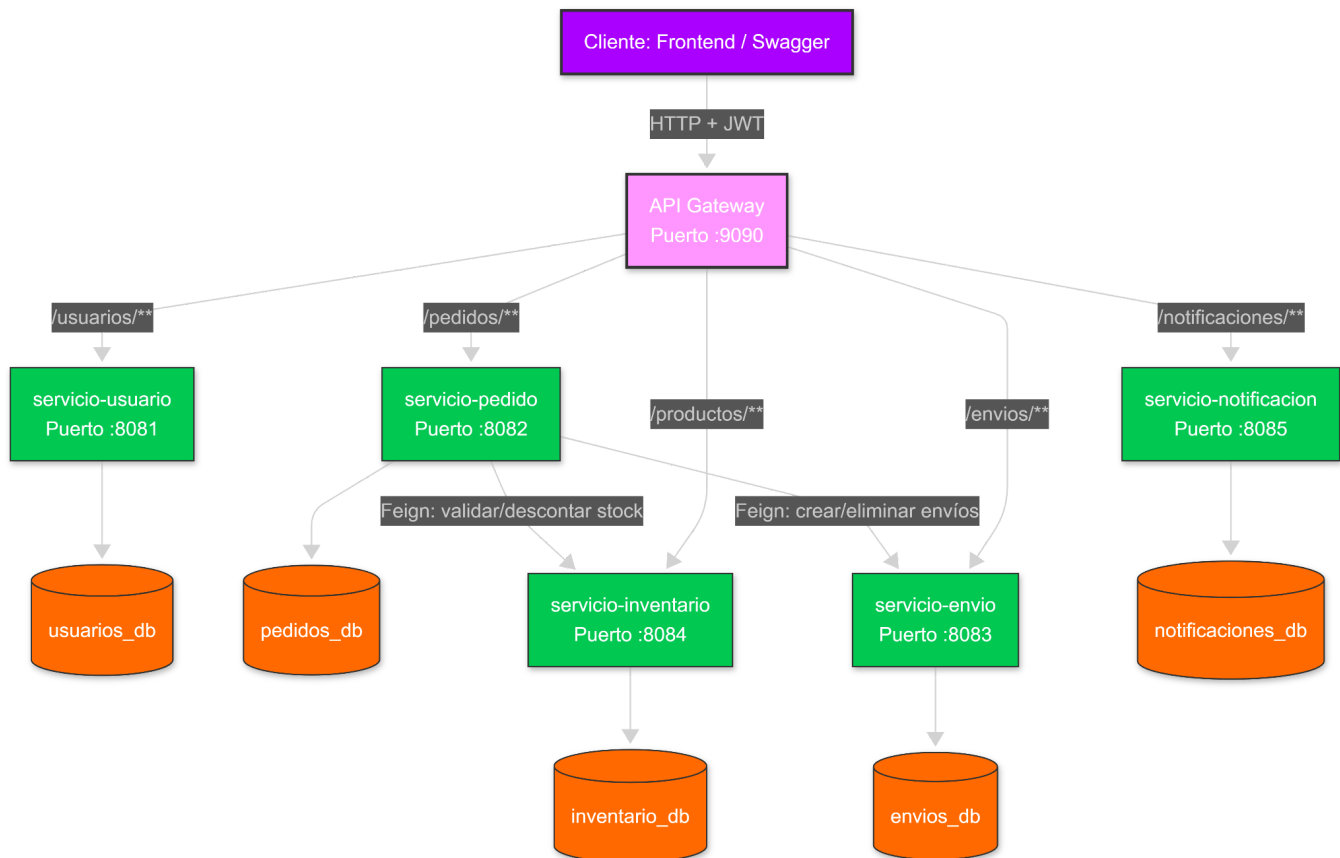
8.4 Lógica automática al crear.

- El estado se fija automáticamente en 'NO_LEIDA' (el cliente no lo envía).
- La fecha se asigna con LocalDateTime.now() en el momento exacto de la inserción.
- El campo mensaje es obligatorio y no puede estar vacío.
- referenciaId permite vincular la notificación a un pedido, envío o producto específico.

9. Flujo de Comunicación entre Servicios.

El siguiente diagrama representa cómo fluye una petición a través del sistema, desde el cliente hasta los microservicios y sus respectivas bases de datos:

*Cambiarlo por un diagrama de flujo



Puntos clave del flujo:

- El cliente NUNCA se comunica directamente con un microservicio; siempre pasa por el Gateway en el puerto 9090.
- El API Gateway valida el JWT antes de redirigir. Si el token es inválido o el rol no tiene permiso, retorna 401/403 sin llegar al servicio.
- Los Feign Clients de Pedidos se comunican internamente usando el nombre del contenedor Docker, no localhost.
- Las bases de datos son privadas; ningún cliente externo puede acceder directamente a ellas.

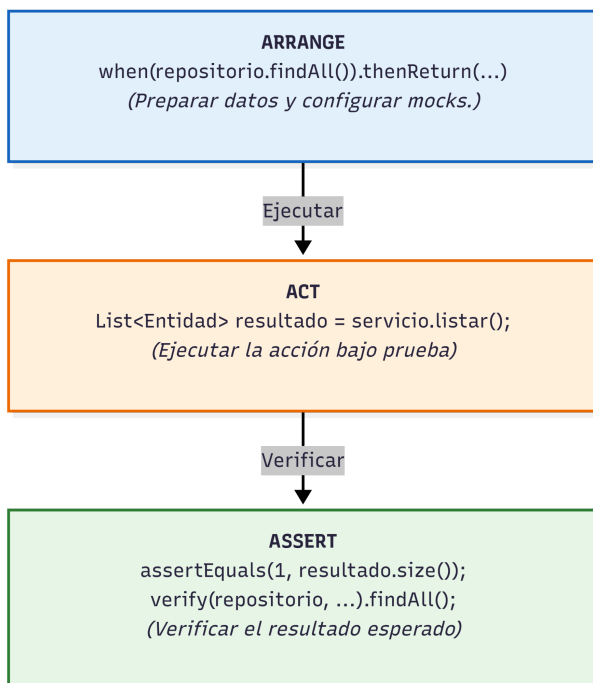
10. Pruebas Unitarias.

Todos los microservicios cuentan con tests unitarios con JUnit 5 y Mockito. Los repositorios y Feign Clients se mockean para probar únicamente la lógica de negocio, sin necesidad de levantar BD ni contenedores Docker.

Servicio	Clase de Test	Tests Incluidos
serviciousuario	UsuarioServiceImplTest	listarUsuarios, guardarUsuarioTest
serviciopedido	PedidoServiceImplTest	listarPedidosTest, guardarPedidoTest
servicioenvio	EnvioServiceImplTest	listarEnviosTest, crearEnvioTest
servicioinventario	ProductoServiceImplTest	listarProductosTest, crearProductoTest
servicionotificacion	NotificacionServiceImplTest	listarNotificacionesTest, crearNotificacionTest

10.1 Patrón Arrange,Act ,Assert.

Todos los tests siguen el patrón AAA. Se usan `@Mock` para repositorios y Feign Clients, e `@InjectMocks` para la clase de servicio:



11. Docker y Contenedores.

11.1 Dockerfiles – Multi-Stage Build.

Todos los microservicios usan el mismo patrón de construcción en dos etapas, reduciendo significativamente el tamaño final de la imagen Docker:

Stage	Imagen Base	Acción
1 build	maven:3.9-eclipse-temurin-21	Compila el proyecto con mvn clean package -DskipTests.
2 runtime	eclipse-temurin:21-jre-alpine	Copia el JAR generado y lo ejecuta (imagen liviana).

11.2 Docker Compose.

El archivo docker-compose.yml levanta los 11 contenedores (6 servicios + 5 bases de datos) en la red interna spring-net:

- Bases de datos: cada una usa mysql:8.0 con MYSQL_ALLOW_EMPTY_PASSWORD.
- Cada BD tiene su propio volumen persistente para sobrevivir a los reinicios.
- Las dependencias están declaradas: el Gateway espera a que los 5 microservicios estén funcionando correctamente..
- En Docker la URL de BD usa el nombre del contenedor (db-usuarios, db-pedidos...) en lugar de localhost.

